

```

-- Databricks notebook source
-- DBTITLE 1,Title
-- MAGIC %md
-- MAGIC # **SQL Bucketing and Segmentation**
-- MAGIC
-- MAGIC **Core Concept:** Bucketing divides continuous values into
categories (Gold, Silver, Bronze) to simplify analysis and support
business decisions.
-- MAGIC
-- MAGIC **Methods Covered:**
-- MAGIC 1. Conditional Logic (CASE WHEN)
-- MAGIC 2. SQL CASE Statement
-- MAGIC 3. Bucketizer-style (CASE WHEN with bucket index)
-- MAGIC 4. Quantile-based Segmentation (PERCENTILE_APPROX)
-- MAGIC 5. Window-based Ranking (PERCENT_RANK)

-- COMMAND -----

-- DBTITLE 1,Load Data
-- MAGIC %md
-- MAGIC ## **1. Load Customer and Order Data**

-- COMMAND -----

-- DBTITLE 1,Load and prepare data
-- Load customer and order data
CREATE OR REPLACE TEMP VIEW customers AS
SELECT * FROM read_files('/Volumes/workspace/default/customers', format
=> 'csv', header => true);

CREATE OR REPLACE TEMP VIEW orders AS
SELECT * FROM
read_files('/Volumes/workspace/default/customers/orders.csv', format =>
'csv', header => true);

-- Calculate total spend per customer and join with customer details
CREATE OR REPLACE TEMP VIEW customer_data AS
SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    c.email,
    c.phone_number,
    c.address,
    c.city,
    c.state,
    c.zip_code,
    COALESCE(SUM(CAST(o.total_amount AS DOUBLE)), 0) AS total_spend,
    COUNT(o.order_id) AS order_count
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email,
c.phone_number, c.address, c.city, c.state, c.zip_code;

-- Display top 10 customers by spend
SELECT customer_id, first_name, last_name, city, total_spend,
order_count
FROM customer_data

```

```

ORDER BY total_spend DESC
LIMIT 10;

-- COMMAND -----

-- DBTITLE 1,Method 1 Header
-- MAGIC %md
-- MAGIC ## **2. Method 1: Conditional Logic (CASE WHEN)**
-- MAGIC
-- MAGIC **Most Common Method** - Uses SQL CASE WHEN to apply business
rules.

-- COMMAND -----

-- DBTITLE 1,Method 1: Conditional logic
-- Method 1: Conditional Logic using CASE WHEN
CREATE OR REPLACE TEMP VIEW df_method1 AS
SELECT
    *,
    CASE
        WHEN total_spend > 100 THEN 'Gold'
        WHEN total_spend BETWEEN 50 AND 100 THEN 'Silver'
        ELSE 'Bronze'
    END AS segment
FROM customer_data;

-- Display segmented customers
SELECT customer_id, first_name, last_name, total_spend, segment
FROM df_method1
ORDER BY total_spend DESC;

-- Segment summary
CREATE OR REPLACE TEMP VIEW segment_counts1 AS
SELECT
    segment,
    COUNT(customer_id) AS customer_count,
    ROUND(SUM(total_spend), 2) AS total_revenue,
    ROUND(SUM(total_spend) / COUNT(customer_id), 2) AS
avg_spend_per_customer
FROM df_method1
GROUP BY segment
ORDER BY total_revenue DESC;

SELECT * FROM segment_counts1;

-- COMMAND -----

-- DBTITLE 1,Method 2 Header
-- MAGIC %md
-- MAGIC ## **3. Method 2: SQL CASE Statement**
-- MAGIC
-- MAGIC **SQL-Based Approach** - Demonstrates the same CASE WHEN logic
(identical to Method 1).

-- COMMAND -----

-- DBTITLE 1,Method 2: SQL CASE

```

```

-- Method 2: SQL CASE Statement (same as Method 1, demonstrates SQL
approach)
CREATE OR REPLACE TEMP VIEW df_method2 AS
SELECT
    *,
    CASE
        WHEN total_spend > 100 THEN 'Gold'
        WHEN total_spend BETWEEN 50 AND 100 THEN 'Silver'
        ELSE 'Bronze'
    END AS segment
FROM customer_data;

-- Display segmented customers
SELECT customer_id, first_name, last_name, total_spend, segment
FROM df_method2
ORDER BY total_spend DESC;

-- Segment summary
CREATE OR REPLACE TEMP VIEW segment_counts2 AS
SELECT
    segment,
    COUNT(customer_id) AS customer_count,
    ROUND(SUM(total_spend), 2) AS total_revenue
FROM df_method2
GROUP BY segment
ORDER BY total_revenue DESC;

SELECT * FROM segment_counts2;

-- COMMAND -----

-- DBTITLE 1,Method 3 Header
-- MAGIC %md
-- MAGIC ## **4. Method 3: Bucketizer-style Segmentation**
-- MAGIC
-- MAGIC **SQL Equivalent** - Mimics MLlib Bucketizer using CASE WHEN
with bucket indices.

-- COMMAND -----

-- DBTITLE 1,Method 3: Bucketizer
-- Method 3: Bucketizer-style segmentation using SQL
-- Note: MLlib Bucketizer is Python-only, this SQL mimics the same
logic
-- Splits: [-inf, 50, 100, +inf] -> buckets 0, 1, 2

CREATE OR REPLACE TEMP VIEW df_method3 AS
SELECT
    *,
    CASE
        WHEN total_spend < 50 THEN 0
        WHEN total_spend < 100 THEN 1
        ELSE 2
    END AS bucket_index,
    CASE
        WHEN total_spend < 50 THEN 'Bronze'
        WHEN total_spend < 100 THEN 'Silver'
        ELSE 'Gold'
    END AS segment

```

```

        END AS segment
FROM customer_data;

-- Display segmented customers
SELECT customer_id, first_name, last_name, total_spend, bucket_index,
segment
FROM df_method3
ORDER BY total_spend DESC;

-- Segment summary
CREATE OR REPLACE TEMP VIEW segment_counts3 AS
SELECT
    segment,
    COUNT(customer_id) AS customer_count,
    ROUND(SUM(total_spend), 2) AS total_revenue
FROM df_method3
GROUP BY segment
ORDER BY total_revenue DESC;

SELECT * FROM segment_counts3;

-- COMMAND -----

-- DBTITLE 1,Method 4 Header
-- MAGIC %md
-- MAGIC ## **5. Method 4: Quantile-based Segmentation**
-- MAGIC
-- MAGIC **Data-Driven Approach** - Splits are determined by data
distribution (33rd and 66th percentiles).

-- COMMAND -----

-- DBTITLE 1,Method 4: Quantile-based
-- Method 4: Quantile-based Segmentation
-- Calculate 33rd and 66th percentiles
CREATE OR REPLACE TEMP VIEW quantile_thresholds AS
SELECT
    PERCENTILE_APPROX(total_spend, 0.33) AS p33,
    PERCENTILE_APPROX(total_spend, 0.66) AS p66
FROM customer_data;

-- Display quantile thresholds
SELECT
    CONCAT('Quantile thresholds: 33rd percentile = ', CAST(p33 AS
STRING), ', 66th percentile = ', CAST(p66 AS STRING)) AS thresholds
FROM quantile_thresholds;

-- Apply quantile-based segmentation
CREATE OR REPLACE TEMP VIEW df_method4 AS
SELECT
    c.*,
    CASE
        WHEN c.total_spend >= q.p66 THEN 'Gold'
        WHEN c.total_spend >= q.p33 AND c.total_spend < q.p66 THEN
'Silver'
        ELSE 'Bronze'
    END AS segment
FROM customer_data c

```

```

CROSS JOIN quantile_thresholds q;

-- Display segmented customers
SELECT customer_id, first_name, last_name, total_spend, segment
FROM df_method4
ORDER BY total_spend DESC;

-- Segment summary
CREATE OR REPLACE TEMP VIEW segment_counts4 AS
SELECT
    segment,
    COUNT(customer_id) AS customer_count,
    ROUND(SUM(total_spend), 2) AS total_revenue
FROM df_method4
GROUP BY segment
ORDER BY total_revenue DESC;

SELECT * FROM segment_counts4;

-- COMMAND -----

-- DBTITLE 1,Method 5 Header
-- MAGIC %md
-- MAGIC ## **6. Method 5: Window-based Ranking**
-- MAGIC
-- MAGIC **Percentile Rank Approach** - Uses percent_rank window
function to assign relative positions.

-- COMMAND -----

-- DBTITLE 1,Method 5: Window ranking
-- Method 5: Window-based Ranking using PERCENT_RANK
CREATE OR REPLACE TEMP VIEW df_method5 AS
SELECT
    *,
    PERCENT_RANK() OVER (ORDER BY total_spend) AS rank_pct,
    CASE
        WHEN PERCENT_RANK() OVER (ORDER BY total_spend) >= 0.66 THEN
'Gold'
        WHEN PERCENT_RANK() OVER (ORDER BY total_spend) >= 0.33 THEN
'Silver'
        ELSE 'Bronze'
    END AS segment
FROM customer_data;

-- Display segmented customers
SELECT customer_id, first_name, last_name, total_spend, rank_pct,
segment
FROM df_method5
ORDER BY total_spend DESC;

-- Segment summary
CREATE OR REPLACE TEMP VIEW segment_counts5 AS
SELECT
    segment,
    COUNT(customer_id) AS customer_count,
    ROUND(SUM(total_spend), 2) AS total_revenue
FROM df_method5

```

```

GROUP BY segment
ORDER BY total_revenue DESC;

SELECT * FROM segment_counts5;

-- COMMAND -----

-- DBTITLE 1,Comparison Header
-- MAGIC %md
-- MAGIC ## **7. Comparison of All Methods**
-- MAGIC
-- MAGIC Compare how different methods distribute customers across
segments.
-- MAGIC
-- MAGIC ### **Practice Tasks**
-- MAGIC 1. â□□ Create Gold/Silver/Bronze segmentation using
conditional logic
-- MAGIC 2. â□□ Group data by segment and count customers
-- MAGIC 3. â□□ Try quantile-based segmentation
-- MAGIC 4. â□□ Compare results of different methods
-- MAGIC 5. â□□ Reflect: which method is most useful and why?

-- COMMAND -----

-- DBTITLE 1,Compare all methods
-- Collect all segment summaries and add method labels
CREATE OR REPLACE TEMP VIEW comparison AS
SELECT '1. Conditional Logic' AS method, segment, customer_count,
total_revenue, avg_spend_per_customer FROM segment_counts1
UNION ALL
SELECT '2. SQL CASE' AS method, segment, customer_count, total_revenue,
NULL AS avg_spend_per_customer FROM segment_counts2
UNION ALL
SELECT '3. Bucketizer' AS method, segment, customer_count,
total_revenue, NULL AS avg_spend_per_customer FROM segment_counts3
UNION ALL
SELECT '4. Quantile-based' AS method, segment, customer_count,
total_revenue, NULL AS avg_spend_per_customer FROM segment_counts4
UNION ALL
SELECT '5. Window Ranking' AS method, segment, customer_count,
total_revenue, NULL AS avg_spend_per_customer FROM segment_counts5;

-- Display comparison
SELECT method, segment, customer_count, total_revenue
FROM comparison
ORDER BY method, total_revenue DESC;

-- Side-by-Side Customer Count Comparison (Pivot)
SELECT
    method,
    SUM(CASE WHEN segment = 'Gold' THEN customer_count ELSE 0 END) AS
Gold,
    SUM(CASE WHEN segment = 'Silver' THEN customer_count ELSE 0 END) AS
Silver,
    SUM(CASE WHEN segment = 'Bronze' THEN customer_count ELSE 0 END) AS
Bronze
FROM comparison
GROUP BY method

```

ORDER BY method;

-- COMMAND -----

```
-- DBTITLE 1,Reflection
-- MAGIC %md
-- MAGIC ## **8. Reflection Questions**
-- MAGIC
-- MAGIC ### **Why do we convert continuous values into categories?**
-- MAGIC * Simplifies analysis and decision-making
-- MAGIC * Makes data more interpretable for business stakeholders
-- MAGIC * Enables targeted marketing and personalized strategies
-- MAGIC * Reduces computational complexity in some scenarios
-- MAGIC
-- MAGIC ### **What is the difference between business segmentation and
technical bucketing?**
-- MAGIC * **Business segmentation**: Based on domain knowledge and
business rules (e.g., Gold > $100)
-- MAGIC * **Technical bucketing**: Based on data distribution
(quantiles, percentiles, statistical methods)
-- MAGIC
-- MAGIC ### **When would fixed thresholds fail?**
-- MAGIC * When data distribution changes over time (inflation, market
shifts)
-- MAGIC * When applying the same thresholds across different
markets/regions
-- MAGIC * When outliers skew the distribution significantly
-- MAGIC * When the business context changes (e.g., economy shifts)
-- MAGIC
-- MAGIC ### **How does quantile-based segmentation differ from fixed
rules?**
-- MAGIC * **Fixed rules**: Same thresholds regardless of data (e.g.,
always $50 and $100)
-- MAGIC * **Quantile-based**: Adapts to data distribution (e.g., top
33% are always Gold)
-- MAGIC * Quantile ensures balanced segment sizes
-- MAGIC * Fixed rules ensure consistent business meaning
-- MAGIC
-- MAGIC ### **Which method would you use in real-world projects?**
-- MAGIC
-- MAGIC **It depends on the use case:**
-- MAGIC
-- MAGIC 1. **CASE WHEN (Conditional Logic)** (Methods 1 & 2)
-- MAGIC * **When**: You have clear business rules and thresholds
-- MAGIC * **Example**: "Premium customers spend over $10,000"
-- MAGIC * **Pros**: Easy to understand, consistent meaning, fast
execution
-- MAGIC * **Cons**: May create imbalanced segments
-- MAGIC
-- MAGIC 2. **Bucketizer-style** (Method 3)
-- MAGIC * **When**: You need explicit bucket indices for downstream
use
-- MAGIC * **Example**: Feature engineering, bucketed joins
-- MAGIC * **Pros**: Provides both numeric index and categorical
label
-- MAGIC * **Cons**: More verbose than simple CASE WHEN
-- MAGIC
-- MAGIC 3. **Quantile-based (PERCENTILE_APPROX)** (Method 4)
```

```

-- MAGIC      * **When**: You want balanced segment sizes
-- MAGIC      * **Example**: "Top 20% of customers"
-- MAGIC      * **Pros**: Adapts to data distribution automatically
-- MAGIC      * **Cons**: Thresholds change as data changes, requires two
passes
-- MAGIC
-- MAGIC 4. **Window Ranking (PERCENT_RANK)** (Method 5)
-- MAGIC      * **When**: You need relative positioning
-- MAGIC      * **Example**: Leaderboards, percentile-based rewards
-- MAGIC      * **Pros**: Precise percentile control, exact rank position
-- MAGIC      * **Cons**: More computationally expensive than CASE WHEN
-- MAGIC
-- MAGIC ---
-- MAGIC
-- MAGIC **Recommendation**: Start with **CASE WHEN** (Method 1) for
most business use cases â it's simple, fast, and easy to maintain.
Use **PERCENTILE_APPROX** (Method 4) when you need balanced segments or
when thresholds are unclear. Use **PERCENT_RANK** (Method 5) when you
need exact percentile rankings for each customer.

```